

Guion de Presentación — GQG System: ABM de Plazos y Módulo de Crédito

Ingeniería de Software III · Primer Examen Final · FP-UNA

Duración estimada: 14-17 min. Cada integrante habla de lo suyo.
Antes de empezar: tener abierto el sistema en `gqg.ianmrc.dev` (o `localhost:3000`),
Adminer para la BD, y el diagrama ER en `azimutt.app` (importando `db/azimutt_schema.sql`).

El sistema se llama **"ABM de PLAZOS"** (así lo nombra la guía del examen). El núcleo de negocio es el **trigger** que genera las cuotas; el ABM de Plazos es donde se configuran las tres modalidades de pago. Ambos hay que mostrarlos en vivo.

[OK] **VERIFICADO contra el código el 2026-06-19**. Se corrió la verificación en Claude Code.
La sección 6 ya refleja solo lo que el código respalda: los [VERIFICAR] se resolvieron (numéricos confirmados; fechas solo obligatorias; trigger completo). No hacer afirmaciones fuera de lo que queda escrito acá.

Mapa de cobertura de la rúbrica

#	Criterio	Pts	Se cubre en	Responsable
1	Comprensión del requerimiento	20	Apertura + Sección 1	Ian
2	Diseño del sistema y base de datos	20	Sección 2 (BD + trigger)	Alberto
3	Prototipo de interfaz gráfica	15	Secciones 3 (ABMs) y 5 (venta/front)	Hugo + Matias G.
4	Implementación técnica (trigger)	15	Sección 2 (demo trigger)	Alberto
5	Presentación oral y argumentación	15	Todo + anexo de preguntas	Todos
6	Trabajo en equipo y coordinación	10	Reparto + fork/PRs	Todos (6)
7	Documentación de apoyo (en el Drive)	5	Subir docs al Drive (ver checklist)	Ian

Las 3 modalidades a nombrar siempre como tres: Contado · Crédito Regular · Crédito Irregular.

Equipo (6): Ian (Jefe/Analista), Alberto (BD/Trigger), Hugo (ABMs + factura imprimible), Diego (Backend/Infra), Matias G. (Venta/Frontend/Diseño), Matias M. (Tester/Calidad).

0. Apertura — Jefe de Proyecto (Ian Delvalle) · ~1.5 min

"Buenas, profesor. Presentamos el sistema ABM de Plazos para GQG System, con el módulo de facturación a crédito.

El problema: GQG System solo facturaba al contado. Con el negocio creciendo, los clientes pedían pagar en cuotas y el sistema no lo permitía — había que rechazar la venta o llevarla a mano, con riesgo de error.

Nuestra solución soporta las **tres modalidades de pago**:

- **Contado (CO):** una sola cuenta, que vence el mismo día de la factura.
- **Crédito Regular:** cuotas mensuales fijas desde la fecha de factura (ej: 30/60/90 días).
- **Crédito Irregular:** el usuario define los días de cada cuota (ej: 30/45/60 días).

El operador configura estas modalidades en el ABM, y al facturar a crédito un trigger en la base de

datos genera las cuotas automáticamente. Es una app full-stack Next.js + MariaDB. Somos seis con roles definidos; cada uno cuenta su parte."

Transición: "Empecemos por cómo entendimos el requerimiento."

1. Comprensión del requerimiento — Jefe de Proyecto / Analista (Ian Delvalle) · ~2 min

-> Criterio 1 (20 pts)

Su parte: acta de constitución, relevamiento, stakeholders, requerimientos funcionales (RF-01 a RF-06) y no funcionales, documento de arquitectura.

"Partimos del requerimiento del cliente: permitir las tres modalidades y que el sistema calcule los vencimientos solo. Todos los datos de la cabecera de la factura ya existían; lo que faltaba era el detalle de cuotas — número, importe, vencimiento, cobrado.

Sobre el alcance: el cliente menciona compras y ventas, pero el entregable pide ensayar el trigger 'para cualquiera de los procesos, compras o ventas'. Implementamos **ventas** (cuentas a cobrar), que es representativo de toda la lógica. Compras es el proceso espejo y se resuelve reutilizando la misma estructura — por eso la tabla de cuentas es polimórfica, ya lo verán.

La solución es parametrizable: si el cliente quiere un plazo nuevo, lo carga en el ABM sin tocar código."

[Mostrar brevemente el ARQUITECTURA.md.]

Transición: "Esto se apoya en el diseño de la base de datos, que es el corazón."

2. Diseño de la base de datos y el trigger — Programador (Alberto Malfitano) · ~3.5 min

-> Criterios 2 (20 pts) y 4 (15 pts) · EL NÚCLEO TÉCNICO

Su parte: modelo de datos completo, justificación de campos, y el trigger ins_ventas.

"Muestro el modelo en el diagrama ER [abrir Azimutt]. Las tablas clave:

- **PLAZOS:** cabecera de la modalidad. cuotas (cantidad) e irregular (0=regular x30 días, 1=irregular con días manuales). Es el núcleo del requerimiento.
- **PLAZO_DETALLES:** solo para irregulares; guarda los días de cada cuota.
- **MONEDAS:** no es decorativa — el trigger usa su campo decimales para truncar el importe de cada cuota. Guaraní=0, Dólar=2. Sin esto habría que hardcodearlo.
- **VENTAS:** le agregamos plazoid (no estaba en el script de cátedra) porque es lo que el trigger necesita para armar las cuotas.
- **CUENTAS_COBRAR:** lo que genera el trigger. Tiene tabla y tabloid en vez de una clave foránea física: es un diseño polimórfico que la hace reusable para ventas y, a futuro, compras, sin duplicar la tabla. La integridad la garantizan la app y el trigger.

Ahora el **trigger** [abrir Adminer], que es el corazón: 1. Valida que el tipo de documento y el plazo coincidan; si no, aborta con SIGNAL SQLSTATE — no se puede registrar un crédito con un plazo de contado. 2. Calcula el importe base: TRUNCATE(total / cuotas, decimales de la moneda). 3. La última cuota = total – base × (cuotas–1): absorbe el redondeo para que la suma sea exacta. Elegimos división simple sin interés, la 'Opción A', que mantiene las cuotas parejas. 4. Calcula el vencimiento con DATE_ADD: regular suma i×30 días; irregular toma los días del detalle. 5. Inserta una fila por

cuota.

Lo demuestro: inserto una venta a crédito irregular... y las cuotas aparecen solas en CUENTAS_COBRAR. Con el caso del cliente: $584.226 \div 3 = 194.742 \times 3 = 584.226$ exacto."

[Demo: INSERT en VENTAS -> filas generadas en CUENTAS_COBRAR.]

Transición: "Estas modalidades, y todos los datos maestros, se administran desde los ABMs que muestra Hugo."

3. ABMs del sistema y factura imprimible — Diseñador / Programador (Hugo) · ~2.5 min

-> Criterio 3 (15 pts)

Su parte: las pantallas de administración (alta/baja/modificación con baja lógica) y la factura imprimible.

"El sistema tiene varios ABMs, todos con la misma lógica de crear, editar y desactivar/ reactivar (baja lógica, no borramos datos):

- **ABM de Plazos** — el que da nombre al sistema. [Demo en vivo] Doy de alta un plazo irregular '30/45/60': elijo crédito, irregular, 3 cuotas, y cargo los días de cada una. Eso crea el registro en PLAZOS y una fila por cuota en PLAZO_DETALLES. Este plazo queda disponible al facturar y el trigger lo usa. (La pantalla de Plazos es alta + listado; la baja lógica de editar/desactivar/reactivar se demuestra en Clientes y Productos.)
- **ABM de Clientes y de Productos** — con baja lógica; los productos llevan precio e IVA.
- **ABM de Depósitos y edición de Empresa** — datos maestros de la factura.

Y al final del circuito, la **factura imprimible**: un comprobante con la liquidación de IVA y el plan de cuotas. [Mostrarla.] Es lo que cierra el flujo completo, de la carga a la salida."

[Demo: alta/edición/baja de un plazo irregular + mostrar la factura imprimible.]

Transición: "Todo esto corre sobre la app que montó Diego."

4. Backend e infraestructura — Programador (Diego Duarte) · ~2 min

-> Criterio 4 (apoyo) y soporte de 3 y 5

Su parte: backend Next.js + MariaDB (mysql2), autenticación por sesión, Docker, setup.sh de un comando, despliegue con nginx + Cloudflare.

"La app es Next.js 15 con TypeScript, conectada a MariaDB. El backend expone los endpoints para los ABMs, las ventas y las cuotas — pero el cálculo de cuotas no está acá, está en el trigger. El backend solo inserta y lee. Eso garantiza que la lógica sea siempre la misma.

Dockerizamos todo: con un comando se levanta la base, Adminer y la app, así todos trabajamos igual. Y está desplegado en producción real en gqg.ianmrc.dev — no es solo local, funciona online."

[Mostrar el sistema corriendo online.]

Transición: "Con una interfaz pensada para el operador, que diseñó Matias."

5. Pantalla de venta, diseño y frontend — Programador / Diseñador (Matias Gaona) · ~2 min

-> Criterio 3 (15 pts)

Su parte: diseño visual (paleta, tipografía, DESIGN.md), pantalla de nueva venta con preview de cuotas en vivo, coherencia visual de todas las pantallas.

"Definimos una dirección de diseño sobria, para un operador que factura muchas horas: paleta monocromática en tonos tinta, claridad, y los números en fuente monoespaciada para que las columnas de importes alineen. Está en el DESIGN.md, así todas las pantallas son coherentes.

La pantalla de nueva venta: selector de cliente, fecha, total, toggle Contado/Crédito que filtra los plazos válidos, y selector de plazo. [Demo] Cuando elijo el plazo, las cuotas se generan al instante — numeradas 01/03, fechas DD/MM/AAAA — con un indicador de que la suma cuadra con el total antes de confirmar. Es el reflejo visual de lo que hace el trigger."

[Demo: cargar venta, cambiar modalidad y plazo, ver cuotas y el indicador de cuadre.]

Transición: "Para que todo sea confiable, hicimos las pruebas."

6. Pruebas, calidad y funcionalidades implícitas — Analista / Tester (Matias Melgarejo) · ~2.5 min

-> Criterio 4 (apoyo) y funcionalidades implícitas

Su parte: casos de prueba de las tres modalidades, verificación del cuadre, validaciones, calidad de código.

"Mi rol fue asegurar que el sistema sea confiable. Probamos las tres modalidades con casos de prueba (están en db/03_pruebas.sql). El caso crítico es el cuadre: que la suma de cuotas dé exactamente el total. Con 584.226 en 3 cuotas, cuadra exacto.

Funcionalidades implícitas que cuidamos:

- **Campos obligatorios:** la venta no se genera si falta cliente, plazo o productos (el total se calcula de los productos; no se tipea a mano).
- **Control de secuencia:** el botón CONFIRMAR VENTA está deshabilitado hasta generar las cuotas; los pasos están ordenados.
- **Baja lógica en los ABM:** en Clientes y Productos no se borran datos, se desactivan y se pueden reactivar.
- **Validación en el trigger:** rechaza un plazo de crédito en un documento de contado, y exige cantidad de cuotas > 0 (ambos con SIGNAL SQLSTATE '45000').

[VERIFICADO contra el código el 2026-06-19 — confirmado, se puede demostrar en vivo:]

- [OK] Los campos numéricos (precio, cantidad, cuotas, días) rechazan letras: filtran los no-dígitos por código y el backend revalida con Number(). OJO: no es type="number" ni zod, no atribuirlo a eso. El **total NO es un campo de texto**: es derivado de los productos.
- [!] Fechas: usan selector nativo (input type="date") y son obligatorias, pero NO hay validación de rango/coherencia. Afirmar solo "fecha obligatoria con selector", nada más.

Y para mantener el código ordenado entre varios, configuramos ESLint, Prettier y Husky, que revisan formato y errores antes de cada commit."

[Demo: dejar un campo obligatorio vacío y mostrar que no deja generar la venta.]

Transición: "Para cerrar, Ian resume."

7. Cierre — Jefe de Proyecto (Ian Delvalle) · ~1 min

-> Criterios 5 y 6

"En resumen: cumplimos los objetivos. El sistema administra plazos y datos maestros con ABMs completos, soporta las tres modalidades, el trigger genera las cuotas automáticamente, hay factura imprimible, y está desplegado y funcionando online. Validamos todo con casos de prueba.

Trabajamos los seis coordinados, sobre un repositorio Git compartido, con cambios vía fork y Pull Requests revisadas y mergeadas — así mantuvimos el control de calidad del código. Toda la documentación de diseño y decisiones está disponible. Quedamos a disposición para preguntas."

Anexo — Posibles preguntas del evaluador y quién responde

Pregunta probable	Responde
"¿Para qué sirve la tabla MONEDAS?"	Alberto — el trigger trunca el importe según sus decimales; no es decorativa.
"¿Por qué CUENTAS_COBRAR no tiene clave foránea física?"	Alberto — diseño polimórfico (tabla/tablaid), reusable para ventas y compras.
"¿Por qué la lógica está en el trigger y no en el código?"	Alberto — confiabilidad: cualquier inserción genera cuotas igual.
"¿Cómo manejan el redondeo de la última cuota?"	Alberto — la última absorbe la diferencia; división simple (Opción A).
"Mostrame cómo doy de alta un plazo nuevo."	Hugo — demo del ABM en vivo.
"¿Cómo cargan los días de un plazo irregular?"	Hugo — en el alta, una fila por cuota en PLAZO_DETALLES.
"¿El cliente no pide módulo de compras?"	Ian — el entregable pide 'cualquiera de los procesos'; compras es el espejo.
"¿Qué pasa si cargo datos inválidos?"	Matias M. — demo de campos obligatorios y control de secuencia.
"¿Por qué Next.js / Docker?"	Diego — mismo entorno para todos, despliegue reproducible.
"¿Cómo coordinaron el trabajo entre seis?"	Cualquiera — fork + Pull Requests revisadas (PRs #1 y #2).
"¿Por qué esa paleta / ese diseño?"	Matias G. — herramienta de uso intensivo: claridad sobre vistosidad.

Checklist antes de presentar

Verificación de validaciones (HECHA el 2026-06-19):

- [x] Verificadas contra el código. Numéricos rechazan letras (confirmado). Fechas: solo obligatorias + selector nativo (sin validación de rango). Trigger: rechaza crédito en contado y exige cuotas > 0 (ambos SIGNAL implementados).
- [x] Sección 6 ajustada: quitados los [VERIFICAR] no confirmados, corregido lo del "total" (es derivado) y el ABM de Plazos (solo alta + listado).

Preparación del entorno:

- [] Sistema corriendo y accesible (probar el login antes).
- [] Adminer abierto con la BD; diagrama ER cargado en Azimutt.
- [] Tener cargado al menos un cliente, productos y un plazo de cada modalidad.
- [] Plan B: capturas del sistema funcionando, por si falla internet.

Ensayo de las demos (los momentos más fuertes):

- [] Alta de un plazo en el ABM de Plazos; baja lógica (desactivar/reactivar) en Clientes/Productos (Hugo).
- [] Factura imprimible (Hugo).
- [] INSERT de venta a crédito -> cuotas generadas por el trigger (Alberto).
- [] Generación de cuotas en vivo + indicador de cuadre en la venta (Matias G.).
- [] Campo obligatorio vacío -> no deja generar la venta (Matias M.).

Documentación (Criterio 7 — 5 pts, NO olvidar):

- [] **Subir la documentación al Drive de la cátedra:** ARQUITECTURA_GQG_System.md, DESIGN.md, INFRA.md, db/01_schema.sql, db/02_trigger.sql, db/03_pruebas.sql, db/azimutt_schema.sql, y el documento de respuestas. Hoy solo está en GitHub.

Reparto (Criterio 6 — 10 pts):

- [] Confirmar que los seis hablan y que el reparto coincide con quién hizo qué de verdad. Importante: verificar que Hugo efectivamente trabajó en los ABMs / factura, o reasignar.